Part C — Big-O Algorithmic Complexity Analysis

Analyze the time and space complexity of the legacy operations. Fill out the analysis table and provide clear mathematical explanations for best, average, and worst cases.

PART C: Big-O Complexity Analysis Table

Operation	Target Code / Function	Best Case	Average Case	Worst Case	Space Complexity	Mathematical Justification
Unamortized Insert	`add_item()`	O(1)	O(1)	O(n)	O(n)	Normal insertion is constant; resize copies n existing elements
Linear Deletion	`remove_item_at()`	O(1)	O(n)	O(n)	O(1)	Deleting the last item needs no shift; deleting near index 0 shifts many items
Linear Search	`findItem()`	O(1)	O(n)	O(n)	O(1)	First element is immediate; last/absent target requires scanning

Array Display	`printAll()`	O(n)	O(n)	O(n)	O(1)	Every valid element must be visited
Matrix Transpose	`processMatrix()`	O(R×C)	O(R×C)	O(R×C)	O(R×C)	Every cell in the R×C matrix is visited

Key mathematical points: insertion is O(1) normally and O(n) during resize; deletion is O(1) at the tail and O(n) near the head; search is O(1) best and O(n) worst; display is O(n); matrix transpose is O(R×C).